Tinjauan Kritis: Secure Control for T–S Fuzzy Wind Turbine Systems Under Hybrid Cyberattacks via an Adaptive Memory Event-Triggered Mechanism

Author(s):	Joko Pawitro
NRP:	6022251005
Lecturer:	Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
Class name:	Artificial Intelligence (EE235271)
Video link:	
Repository link:	Repo

Abstract

Review ini mengevaluasi secara kritis metodologi kendali aman yang diusulkan Xu dkk. [1] untuk sistem turbin angin berbasis Permanent Magnet Synchronous Generator (PMSG) di bawah serangan siber hibrida.

Fokus analisis terletak pada kebaruan Adaptive Memory Event-Triggered Mechanism (AMETM) yang mengintegrasikan sinyal acknowledgement (ACK) untuk deteksi Denial-of-Service (DoS), serta formulasi stabilitas berbasis Lyapunov dan optimasi Linear Matrix Inequalities (LMI).

Ulasan menunjukkan bahwa mekanisme pemicu adaptif dengan memori historis mampu menjaga mean-square H_∞ asymptotic stability pada kondisi normal, serta uniformly ultimate boundedness saat terjadi DoS.

Review juga membahas implikasi praktis, batasan implementasi, dan relevansinya terhadap kurikulum AI S2 ITS.

Kata kunci: T–S fuzzy systems; AMETM; cyberattacks; wind turbine control; H_∞ stability.

Pendahuluan

Dalam evolusi smart grid, integrasi energi terbarukan menghadapi dua tantangan utama: non-linearitas tinggi pada dinamika sistem fisik dan kerentanan komunikasi terhadap gangguan siber.

Turbin angin berbasis PMSG merupakan teknologi penting karena efisiensi dan kebutuhan perawatan yang relatif rendah, tetapi sifat stokastik kecepatan angin membuat kendali klasik sulit menjaga performa secara konsisten.

Pada konteks tersebut, logika fuzzy – khususnya model Takagi–Sugeno (T–S) – relevan karena mampu memetakan sistem non-linear ke bentuk subsistem linear untuk desain kontroler yang lebih robust [2].

Permasalahan menjadi lebih kompleks ketika Networked Control Systems (NCS) menggunakan kanal nirkabel. Serangan DoS menurunkan ketersediaan data, sedangkan deception attacks merusak integritas data.

Tanpa strategi proteksi adaptif, kedua serangan tersebut dapat memicu degradasi performa bahkan ketidakstabilan mekanis pada turbin.

Tujuan review ini adalah mengevaluasi efektivitas AMETM dan T–S fuzzy dalam menjaga ketahanan sistem di bawah serangan hibrida, serta menganalisis keterkaitannya dengan capaian pembelajaran mata kuliah Artificial Intelligence (EE235271) [3].

Identitas Jurnal dan Informasi Umum

Komponen Identitas	Detail Informasi
Judul Artikel	Secure Control for T–S Fuzzy Wind Turbine Systems Under Hybrid Cyberattacks via an Adaptive Memory Event-Triggered Mechanism
Penulis	Dong Xu, Yajuan Liu, Sangmoon Lee
Nama Jurnal	IEEE Transactions on Fuzzy Systems
Volume/Issue/Tahun	Vol. 33, No. 5, 2025
DOI	10.1109/TFUZZ.2025.3525778
Afiliasi Penulis	North China Electric Power University (China) dan Kyungpook National University (Korea Selatan)

Artikel ini terbit pada jurnal bereputasi tinggi di bidang fuzzy systems. Karena itu, kontribusi metodenya layak dianalisis lebih dalam untuk konteks kontrol aman sistem energi.

Ringkasan Artikel: Dekonstruksi Metodologi

Artikel ini menargetkan masalah kontrol aman untuk turbin angin yang dimodelkan dengan Takagi–Sugeno fuzzy models ketika sistem menghadapi hybrid cyberattacks (gabungan DoS dan deception).

Kerangka yang diusulkan menyatukan pemodelan fuzzy, mekanisme pemicu adaptif, dan jaminan stabilitas stokastik:

1. **Pemodelan T–S fuzzy dan variabel premis.** Dinamika non-linear PMSG direpresentasikan sebagai beberapa subsistem linear menggunakan membership functions. Variabel premis utama adalah kecepatan sudut rotor (ω_g), lalu kontroler didesain melalui Parallel Distributed Compensation (PDC).
2. **Adaptive Memory Event-Triggered Mechanism (AMETM).** Mekanisme menyimpan data historis rilis sinyal, kemudian menyesuaikan ambang pemicu secara adaptif. Integrasi sinyal ACK menjadi elemen penting karena dapat mengidentifikasi kondisi DoS untuk menahan transmisi sia-sia dan menghemat sumber daya komunikasi.
3. **Zero-Order Hold saat DoS.** Ketika kanal komunikasi terputus, Zero-Order Hold (ZOH) mempertahankan aksi kontrol terakhir yang valid sehingga loop kontrol tidak langsung kehilangan kendali.
4. **Jaminan stabilitas.** Penulis menurunkan kondisi berbasis LMI yang menjamin mean-square H_∞ asymptotic stability pada kondisi normal, serta uniformly ultimate boundedness pada interval DoS.

Analisis Kritis: Evaluasi dan Dampak

Kelebihan

- **Diferensiator teknis jelas.** Penggunaan memori historis dalam AMETM meningkatkan respons pada kondisi transien dibandingkan event-triggering statis.
- **Deteksi serangan lebih praktis.** Integrasi ACK memberi indikator DoS tanpa perlu perangkat intrusion detection terpisah.
- **Landasan matematis kuat.** Pembuktian berbasis Lyapunov dan LMI memberikan justifikasi formal terhadap klaim kestabilan.
- **Efisiensi komunikasi.** Penurunan tingkat rilis paket saat saluran diserang bukan kelemahan, tetapi strategi untuk mencegah pemborosan energi node sensor.

Kekurangan dan Keterbatasan

- **Kompleksitas desain tinggi.** Penentuan gain kontroler memerlukan penyelesaian LMI yang kompleks pada fase perancangan.
- **Asumsi bounded uncertainty.** Efek serangan dimodelkan dalam batas ketidakpastian tertentu; pada skenario serangan tak terduga, asumsi ini bisa terlalu konservatif.
- **Overhead memori dan perangkat.** Pendekatan memory-based menambah kebutuhan buffer di node, berpotensi menaikkan biaya perangkat lapangan.
- **Validasi lapangan terbatas.** Mayoritas hasil masih berbasis simulasi, belum diuji luas melalui Hardware-in-the-Loop atau implementasi operasional.

Rekomendasi Perbaikan

- Optimasi parameter fuzzy dan bobot matriks desain menggunakan pendekatan meta-heuristik (misalnya genetic algorithm).
- Uji ketahanan model terhadap skenario unbounded deception attacks untuk memetakan batas kegagalan absolut.
- Lakukan validasi Hardware-in-the-Loop (HIL) untuk mengukur latensi pemrosesan, beban komputasi, dan performa real-time.

Relevansi dengan Kurikulum AI S2 ITS

Mata kuliah Artificial Intelligence (EE235271) di ITS (lihat kurikulum AI [3]) mencakup topik fuzzy system, neural networks, dan optimisasi.

Artikel yang direview memiliki keterkaitan langsung dengan beberapa capaian pembelajaran tingkat magister, terutama pada aspek analisis, desain sistem cerdas, dan pengambilan keputusan berbasis data.

Secara spesifik:

- **CPMK / CLO 3 — Penguasaan fuzzy systems (Capaian Pembelajaran Mata Kuliah nomor 3).** Implementasi T–S fuzzy pada PMSG menunjukkan aplikasi nyata logika fuzzy untuk sistem dinamis nonlinear. Berdasarkan kurikulum AI ITS, mata kuliah ini mengaitkan diri pada program outcomes **CPL-04 (PLO-04)** dan **CPL-08 (PLO-08)**.
- **CPL-04 (PLO-04) — Analisis dan desain rekayasa.** Strategi kendali aman yang diusulkan memperlihatkan integrasi teori kontrol, komunikasi, dan keamanan siber, sehingga relevan dengan CPL-04 yang menekankan penguasaan konsep dan prinsip rekayasa.
- **CPL-08 (PLO-08) — Pengambilan keputusan teknologi.** Mekanisme adaptif berbasis kondisi serangan menggambarkan pengambilan keputusan operasional yang efisien dan berorientasi keberlanjutan, sesuai dengan CPL-08 yang berkaitan dengan kemampuan pengambilan keputusan berbasis analisis data.

Dalam konteks Indonesia, pendekatan ini relevan untuk sistem PLTB karena profil angin pesisir yang turbulen dapat memicu ketidaksinkronan variabel premis antara sensor dan kontroler.

Karena itu, AMETM berpotensi menjadi topik proyek riset S2 yang menggabungkan analisis teoritis, simulasi, dan validasi implementasi.

Kesimpulan dan Rekomendasi Strategis

- Integrasi AMETM dan T–S fuzzy merupakan kontribusi signifikan untuk meningkatkan ketahanan kontrol turbin angin terhadap serangan siber hibrida.
- Diferensiator utama artikel adalah pemanfaatan memori historis untuk meningkatkan kualitas keputusan pemicu komunikasi, bukan sekadar menurunkan frekuensi transmisi.
- Untuk memperkuat adopsi industri, penelitian lanjutan perlu menitikberatkan pada optimasi parameter otomatis, skenario serangan yang lebih agresif, dan validasi HIL [4].

Catatan personal:

Berdasarkan pengalaman profesional penulis pada monitoring dan pengendalian sistem tenaga terdistribusi dan pengolahan data sensor, pendekatan AMETM menarik karena sejalan dengan praktik penghematan komunikasi dan kebutuhan keandalan di lapangan.

Namun, dari perspektif integrasi lapangan (offshore/subsea cable monitoring), aspek robustness terhadap kondisi lingkungan non-ideal dan maintenance overhead perlu perhatian khusus.

Dalam konteks subsea cable monitoring berbasis DTS/DAS, keterkaitan AMETM dapat diringkas sebagai berikut:

- Menjaga kualitas keputusan saat komunikasi tidak stabil (packet loss, latensi tinggi, dan gangguan telemetri).
- Kombinasi ACK dan memori historis membantu membedakan kondisi serangan/putus komunikasi dari fluktuasi operasional normal.
- Strategi event-triggered menekan beban transmisi sehingga kanal komunikasi lebih efisien tanpa kehilangan informasi penting.
- Implementasi tetap memerlukan desain yang mempertimbangkan keterbatasan buffer, daya komputasi edge, dan kompleksitas maintenance perangkat bawah laut.

References

[1] D. Xu, Y. Liu, and S. Lee, "Secure control for t–s fuzzy wind turbine systems under hybrid cyberattacks via an adaptive memory event-triggered mechanism," *IEEE Transactions on Fuzzy Systems*, vol. 33, no. 5, pp. 1475–1487, 2025.

[2] T. J. Ross, *Fuzzy Logic with Engineering Applications*, 3rd ed. Chichester, U.K.: John Wiley & Sons, 2010.

[3] Institut Teknologi Sepuluh Nopember, "Kurikulum artificial intelligence (ee235271)," Institut Teknologi Sepuluh Nopember, Tech. Rep., 2026, course description and learning outcomes; local copy: kurikulum_ai.pdf.

[4] E. Tian and C. Peng, "Memory-based event-triggering H_∞ load frequency control for power systems under deception attacks," *IEEE Transactions on Cybernetics*, vol. 50, no. 11, pp. 4610–4618, Nov 2020.

Appendix A. Laporan Implementasi Python Prototype

Lampiran ini merangkum implementasi Python untuk prototype AMETM T–S fuzzy secara ringkas: arsitektur proyek, fungsi inti, serta validasi berbasis unit test dan pemeriksaan manual.

A.1 Struktur Proyek Implementasi

Listing 1: Struktur direktori utama implementasi Python tugas 4

```
folder_proyek/
|-- pyproject.toml
|-- src/
|   `-- ametm_ts/
|       |-- cli.py
|       |-- fuzzy.py
|       |-- ametm.py
|       |-- simulation.py
|       |-- metrics.py
|       `-- wind_turbine.py
|-- tests/
|   |-- test_fuzzy.py
|   |-- test_ametm.py
|   |-- test_data_and_simulation.py
|   `-- test_metrics.py
|-- data/
|   |-- synthetic_wind_turbine_case.csv
|   `-- simulation_results.csv
`-- plots/
    `-- simulation_overview.png
```

A.2 Alur Eksekusi Pipeline

Pipeline pada `src/ametm_ts/cli.py` berjalan dalam urutan:

1. Membuat data sintetik jika file input belum tersedia.
2. Menjalankan simulasi plant + kontrol event-triggered adaptif.
3. Menyimpan hasil simulasi ke CSV.
4. Membuat visualisasi ringkasan.
5. Menghitung metrik evaluasi.

A.3 Snippet Fungsi Utama

1) Bobot fuzzy dua aturan (`fuzzy.py`)

Listing 2: compute_two_rule_weights(...)

```python
def compute_two_rule_weights(rotor_speed, nominal_speed=12.0, spread=4.0):
    if spread <= 0:
        raise ValueError("spread must be positive")

    left = nominal_speed - spread / 2.0
    right = nominal_speed + spread / 2.0

    if rotor_speed <= left:
        low = 1.0
    elif rotor_speed >= right:
        low = 0.0
    else:
        low = (right - rotor_speed) / (right - left)

    high = 1.0 - low
    weights = np.array([low, high], dtype=float)
    weights /= np.sum(weights)
    return weights
```

2) Ambang adaptif AMETM (`ametm.py`)

Listing 3: adaptive_threshold(...) dan should_release(...)

```
def adaptive_threshold(self, ack_attack_flag: int) -> float:
    history_mean = float(np.mean(self._history))
    attack_term = self.attack_scale if int(ack_attack_flag) == 1 else 0.0
    return self.base_threshold * (1.0 + attack_term + self.memory_weight *
        history_mean)

def should_release(self, state_error, ack_attack_flag: int) -> bool:
    threshold = self.adaptive_threshold(ack_attack_flag)
    error_energy = float(state_error.T @ state_error)
    return error_energy > threshold
```

3) Logika event-triggered + ZOH saat DoS (`simulation.py`)

Listing 4: Ekstrak loop utama run_simulation(...)

```
release_attempt = trigger.should_release(
    state_error=(state_before - last_released_state),
    ack_attack_flag=ack_flag,
)

dos_active = dos_flag == 1
release_success = bool(release_attempt and not dos_active)

if release_success:
    held_control = control_candidate
    last_released_state = state_before.copy()

trigger.update_memory(release_success)
state_after = plant.step(float(held_control), disturbance, weights)
```

4) Ringkasan metrik evaluasi (`metrics.py`)

Listing 5: summarize_metrics(...)

```
state_norm = np.sqrt(results["x1"] ** 2 + results["x2"] ** 2)
release_ratio = float(results["release_success"].mean())
bounded_ratio = float((state_norm <= state_bound).mean())

return {
    "release_ratio": release_ratio,
    "communication_saving": 1.0 - release_ratio,
    "bounded_ratio": bounded_ratio,
    "rms_state": float(np.sqrt(np.mean(state_norm**2))),
    "control_effort_l1": float(np.mean(np.abs(results["u_applied"]))),
}
```

5) Orkestrasi CLI (`cli.py`)

Listing 6: Ekstrak run_pipeline(...)

```
if not data_path.exists():
    synthetic = generate_synthetic_case(num_steps=num_steps, dt=dt, seed=seed)
    save_case_csv(synthetic, data_path)

case_data = load_case_csv(data_path)
results = run_simulation(case_data, plant=plant, trigger=trigger, gains=gains)
save_case_csv(results, result_path)
plot_simulation_results(results, plot_path)

return summarize_metrics(results)
```

A.4 Validasi Berbasis Code (Unit Test)

Pengujian otomatis dilakukan dengan `unittest` pada empat berkas:

- `tests/test_fuzzy.py`: validasi normalisasi bobot fuzzy dan batas ekstrem.
- `tests/test_ametm.py`: validasi kenaikan threshold pada status serangan dan keputusan release.
- `tests/test_data_and_simulation.py`: validasi skema data input dan konsistensi output simulasi.
- `tests/test_metrics.py`: validasi rentang metrik dan ketersediaan key output.

Perintah uji:

Listing 7: Eksekusi unit test

```
python -m unittest discover -s tests -t . -p "test_*.py"
```

Ringkasan hasil eksekusi:

Listing 8: Output unit test

```
.......
----------------------------------------------------------------------
Ran 7 tests in 0.006s

OK
```

A.5 Validasi Manual Hasil Simulasi

Selain unit test, dilakukan verifikasi manual terhadap metrik dari file `data/simulation_results.csv`. Konsistensi utama dicek dengan relasi:

$$\texttt{communication_saving} = 1 - \texttt{release_ratio}$$

Hasil verifikasi manual sesuai output pipeline:

Metrik	**Nilai**
release_ratio	0.010000
communication_saving	0.990000
bounded_ratio	1.000000
rms_state	0.046057
control_effort_l1	0.025128

Pemeriksaan manual tambahan pada membership dua aturan menunjukkan normalisasi konsisten ($w_{low} + w_{high} = 1$):

Kecepatan Rotor	w_{low}	w_{high}	Jumlah
8.0	1.0000	0.0000	1.0000
12.0	0.5000	0.5000	1.0000
16.0	0.0000	1.0000	1.0000

A.6 Ringkasan Lampiran

Implementasi Python telah dipisahkan per modul fungsional (fuzzy, AMETM, simulasi, metrik, dan CLI), dilengkapi validasi unit test, serta diverifikasi manual pada metrik kunci. Struktur ini membuat prototype mudah diuji ulang, dikembangkan, dan dipetakan kembali ke konsep teoretis yang dibahas pada review utama.